

Interfaces e o Padrão Strategy em Java

Interfaces são contratos. Strategy é o que você constrói quando um contrato precisa de comportamento trocável em tempo de execução.

Parte 1: Interfaces em Java

1.1 Um problema para começar

Uma loja quer aceitar várias formas de pagamento:

- **Boleto** – gera um código de barras e um vencimento;
- **Cartão de Crédito** – processa a cobrança em uma operadora, podendo parcelar;
- **Pix** – gera uma chave e confirma a transferência na hora.

Apesar de serem bem diferentes entre si, todas essas formas de pagamento precisam fazer a mesma coisa: **processar um pagamento**.

1.2 Por que herança não resolve

Já sabemos que, quando classes compartilham comportamento, podemos usar herança. Mas será que faz sentido dizer que `Boleto extends CartaoCredito`? Ou que `Pix extends Boleto`?

Pense bem...

Boleto, Cartão de Crédito e Pix **não são tipos especializados um do outro**. Eles não compartilham atributos nem comportamento em comum – exceto uma coisa: todos processam um pagamento.

Herança representa uma relação "**é um**". Aqui, essa relação simplesmente não existe entre essas classes.

1.3 A armadilha da superclasse comum

Uma ideia tentadora seria criar uma superclasse `Pagamento`:

```

1 public abstract class Pagamento {
2     public abstract void pagar(double valor);
3 }
4
5 public class Boleto extends Pagamento { /* ... */ }
6 public class CartaoCredito extends Pagamento { /* ... */ }
7 public class Pix extends Pagamento { /* ... */ }

```

Listing 1: Uma solução que parece funcionar

Esse código **compila e funciona**. É por isso que essa armadilha é mais traiçoeira que a primeira: o problema não está na sintaxe, está no **significado**. Um **Pagamento** é um **evento** (acontece em uma data, tem um valor, tem um status); **Boleto**, **CartaoCredito** e **Pix** não são esse evento – são os **meios** usados para realizá-lo. A frase certa não é "Boleto é um Pagamento", e sim "Boleto **processa** um pagamento".

Mesmo funcionando, herança seria a ferramenta errada

- a superclasse não teria nenhum atributo nem método com corpo compartilhado – existiria só para forçar um tipo em comum;
- em Java, uma classe só pode ter **extends** de uma superclasse. Se **Boleto** também precisasse ser um **Documento**, essa herança já estaria “gasta” com **Pagamento**;
- se alguém tentasse aproveitar a superclasse para algo além da assinatura (como imprimir um comprovante), o único jeito seria encher **Pagamento** de **instanceof** – um sinal claro de que a classe não sabe lidar com suas próprias subclasses de forma genérica.

Não precisamos que essas classes compartilhem **código**. Precisamos apenas garantir que todas elas **prometam** ter um método para processar o pagamento. Essa ferramenta existe em Java e se chama **interface**.

1.4 O que é uma interface?

Definição

Uma **interface** é um tipo que define um conjunto de métodos que uma classe promete implementar, sem dizer **como** esses métodos devem ser implementados. Por isso dizemos que uma interface é um **contrato**: ela declara **o que** deve ser feito, e cada classe que a implementa decide **como** fazer.

Se a classe assina o contrato (**implements**) e não cumpre uma das obrigações, o código **não compila**.

1.5 Sintaxe e um primeiro exemplo

Uma interface é declarada com a palavra-chave **interface**, no lugar de **class**:

```

1 public interface Pagavel {
2     void pagar(double valor);

```

3 }

Listing 2: A interface Pagavel

O método pagar não tem corpo – apenas assinatura. Toda classe que implementar Pagavel será **obrigada** a fornecer uma implementação para pagar(double valor).

1.6 Implementando o contrato: Boleto, Cartão e Pix

Uma classe assina o contrato com a palavra-chave implements:

```
1 public class Boleto implements Pagavel {
2     private String codigoBarras;
3
4     public Boleto(String codigoBarras) {
5         this.codigoBarras = codigoBarras;
6     }
7
8     @Override
9     public void pagar(double valor) {
10         System.out.println("Pagando R$ " + valor
11             + " via boleto " + codigoBarras);
12     }
13 }
14
15 public class CartaoCredito implements Pagavel {
16     private String numero;
17
18     public CartaoCredito(String numero) {
19         this.numero = numero;
20     }
21
22     @Override
23     public void pagar(double valor) {
24         System.out.println("Cobrando R$ " + valor
25             + " no cartão " + numero);
26     }
27 }
28
29 public class Pix implements Pagavel {
30     private String chave;
31
32     public Pix(String chave) {
33         this.chave = chave;
34     }
35
36     @Override
37     public void pagar(double valor) {
38         System.out.println("Transferindo R$ " + valor
39             + " via Pix " + chave);
40     }
41 }
```

Listing 3: As três formas de pagamento implementando Pagavel

1.7 Regras básicas de uma interface

- os métodos de uma interface são, por padrão, **públicos e abstratos** (sem corpo);
- uma interface **não possui atributos de instância** – não guarda estado como uma classe normal;
- uma classe usa **implements** (em vez de **extends**) para assinar o contrato;
- se a classe não implementar **todos** os métodos, o código não compila.

Diferença importante em relação a herança

Herança (**extends**) representa "é um". Interface (**implements**) representa "sabe fazer" ou "cumpre este contrato".

1.8 Uma classe, várias interfaces

Diferente da herança de classes (onde só existe uma superclasse direta), uma classe pode assinar **vários contratos ao mesmo tempo**:

```
1 public interface Cancelavel {
2     void cancelar();
3 }
4
5 public class CartaoCredito implements Pagavel, Cancelavel {
6     @Override
7     public void pagar(double valor) { /* ... */ }
8
9     @Override
10    public void cancelar() { /* ... */ }
11 }
```

Listing 4: CartaoCredito cumprindo dois contratos

1.9 Polimorfismo com interfaces

Assim como acontece com herança, uma variável do tipo da interface pode referenciar qualquer classe que a implemente:

```
1 List<Pagavel> formas = new ArrayList<>();
2 formas.add(new Boleto("00190.00009"));
3 formas.add(new CartaoCredito("1234-5678"));
4 formas.add(new Pix("loja@email.com"));
5
6 for (Pagavel forma : formas) {
7     forma.pagar(100.0);
8 }
```

Listing 5: Mesmo tipo de variável, comportamentos diferentes

O **for** nem sabe (nem precisa saber) se cada **forma** é um boleto, um cartão ou um Pix. Ele só precisa saber que todo **Pagavel** tem um método **pagar()**.

1.10 Programar para a abstração, não para a implementação

Princípio

Sempre que possível, escreva código que dependa de **interfaces** (o que um objeto sabe fazer), e não de **classes concretas** (como ele faz por dentro).

```
1 List<String> lista = new ArrayList<>();
2 // ABSTRAÇÃO: List -> o que a variável promete fazer
3 // IMPLEMENTAÇÃO: ArrayList -> como isso é feito por dentro
```

Listing 6: Abstração vs. implementação

Se um método recebe `List` como parâmetro (e não `ArrayList`), ele funciona com **qualquer** implementação: `ArrayList`, `LinkedList` ou `Vector`. Trocar a implementação escolhida na criação do objeto não exige alterar mais nenhuma linha do resto do código.

E se eu precisar de algo que só o `ArrayList` tem?

Não é má prática declarar `ArrayList` direto quando você realmente precisa de um método exclusivo dele (como `ensureCapacity`). A regra é simples: se **todos** os métodos que você usa estão em `List`, declare `List`; se **pelo menos um** só existe em `ArrayList`, declare `ArrayList` direto. O que **não** se deve fazer é esconder essa dependência atrás de um `cast` (`((ArrayList<String>) lista)`) – isso finge que o código só precisa de `List`, quando na prática precisa de `ArrayList`.

Curiosidade: o mesmo padrão se repete em toda a Java Collections Framework. Não é só a `List` que esconde várias implementações atrás do mesmo contrato – isso se repete o tempo todo, e é um ótimo jeito de fixar a ideia de "programar para a abstração".

```
1 List<String> nomes1 = new ArrayList<>();
2 List<String> nomes2 = new LinkedList<>();
3 List<String> nomes3 = new Vector<>();
4
5 nomes1.add("Ana");
6 nomes2.add("Ana");
7 nomes3.add("Ana");
8 // as tres linhas de add() fazem exatamente a mesma coisa
9 // do ponto de vista de quem esta chamando o metodo
```

Listing 7: Três implementações diferentes, exatamente a mesma operação

Por dentro, cada uma se comporta diferente

- **ArrayList** – pense em uma prateleira numerada: pegar o item na posição 5 é instantâneo, mas inserir um item bem no meio empurra todo mundo uma casa pra frente;
- **LinkedList** – pense em uma corrente de elos, cada um apontando para o próximo: inserir ou remover nas pontas é rápido, mas para pegar o item na posição 5 é preciso "andar" elo por elo até lá;

- **Vector** – por dentro é quase idêntico ao `ArrayList`, mas com sincronização extra pensada para várias *threads* usando a lista ao mesmo tempo. Essa sincronização tem um custo de desempenho que a imensa maioria dos programas não precisa pagar.

Regra prática

Na dúvida, comece com `ArrayList` – é a escolha certa na grande maioria dos casos. Só troque para `LinkedList` se o programa realmente fizer muitas inserções/remoções no meio ou nas pontas da lista. E praticamente nunca use `Vector` em código novo: ele é considerado uma classe legada (antiga), e `ArrayList` faz o mesmo trabalho sem o custo extra da sincronização.

Set: HashSet, LinkedHashSet e TreeSet. A interface `Set` representa "uma coleção que não aceita elementos duplicados":

```
1 Set<String> visitantes = new HashSet<>();  
2 visitantes.add("Carlos");  
3 visitantes.add("Carlos"); // ignorado, Set nao aceita duplicados  
4  
5 System.out.println(visitantes.size()); // 1
```

Listing 8: Set garante que não existam duplicados

- **HashSet** – não garante nenhuma ordem; é a implementação mais rápida para simplesmente checar "esse elemento já existe?";
- **LinkedHashSet** – também não aceita duplicados, mas lembra a ordem em que os elementos foram inseridos;
- **TreeSet** – mantém os elementos sempre ordenados automaticamente, ao custo de ser um pouco mais lento que o `HashSet`.

Map: HashMap, LinkedHashMap e TreeMap. A interface `Map` representa "uma coleção de pares chave-valor":

```
1 Map<String, Double> precos = new HashMap<>();  
2 // Map<String, Double> precos = new TreeMap<>();  
3  
4 precos.put("Caneta", 2.50);  
5 precos.put("Caderno", 12.90);  
6  
7 System.out.println(precos.get("Caneta")); // 2.5
```

Listing 9: Trocar `HashMap` por `TreeMap` não muda o resto do código

- **HashMap** – a mais usada no dia a dia, sem ordem garantida entre as chaves;
- **LinkedHashMap** – lembra a ordem em que as chaves foram inseridas;

- **TreeMap** – mantém as chaves sempre ordenadas.

Interface	Implementação	Ordem?	Duplicados?	Quando usar
List	ArrayList	de inserção	permite	escolha padrão
List	LinkedList	de inserção	permite	muitas inserções/remoções nas pontas
List	Vector	de inserção	permite	quase nunca (legado)
Set	HashSet	nenhuma	não permite	checagem rápida de existência
Set	LinkedHashSet	de inserção	não permite	sem duplicados, lembrando a ordem
Set	TreeSet	ordenada	não permite	sem duplicados, sempre ordenado
Map	HashMap	nenhuma	chaves únicas	escolha padrão
Map	LinkedHashMap	de inserção	chaves únicas	lembrar a ordem de inserção
Map	TreeMap	ordenada por chave	chaves únicas	chaves sempre ordenadas

A mesma lição, em toda parte

Isso é exatamente o polimorfismo por interface que vimos com **Pagavel**: várias classes diferentes cumprindo o mesmo contrato, cada uma com sua própria forma de fazer por dentro. Programar para a abstração (**List**, **Set**, **Map**) em vez da implementação (**ArrayList**, **HashSet**, **HashMap**) é o que permite trocar essas peças sem quebrar o resto do sistema.

1.11 Nomeando interfaces

O nome de uma interface é a primeira – e às vezes única – documentação do contrato que ela representa. Existem duas convenções comuns:

- **Adjetivos de capacidade**, terminados em **-vel** (equivalente a **-able**): **Pagavel**, **Comparable**, **Runnable**, **Serializable**. Descrevem algo que a classe promete **saber fazer**;
- **Substantivos de papel**: **List**, **Comparator**, **EstrategiaXp**. Usados quando a interface representa **uma coisa** ou **um papel** dentro do sistema – muito comum em interfaces usadas no padrão **Strategy**, como veremos na Parte 2.

Erros comuns de nomenclatura

Prefixo **I** (**IPagavel**, convenção de outras linguagens); sufixo redundante (**PagavelInterface**); nomes genéricos demais (**Processavel**, **Gerenciavel**) que não dizem nada sobre o domínio.

Verifique seu entendimento

- Por que `Boleto` `extends` `Pagamento` compila, mas ainda assim é uma escolha de projeto ruim?
- Qual é a diferença entre o que `extends` representa e o que `implements` representa?
- Por que `Pagavel` é um bom nome de interface, seguindo a mesma lógica de `Runnable`?

Parte 2: O Padrão Strategy

2.1 Quando o comportamento precisa mudar em tempo de execução

Até agora, escolhemos a forma de pagamento **na hora de criar o objeto**. Mas e se um `Pedido` precisar começar com uma forma de pagamento, e o cliente puder **trocar de ideia** antes de finalizar a compra?

Precisamos de um objeto que guarde uma **referência** para um `Pagavel`, e permita **trocar** essa referência a qualquer momento.

2.2 Construindo o Pedido

```
1 public class Pedido {
2     private double valorTotal;
3     private Pagavel formaPagamento;
4
5     public Pedido(double valorTotal, Pagavel formaPagamento) {
6         this.valorTotal = valorTotal;
7         this.formaPagamento = formaPagamento;
8     }
9
10    public void setFormaPagamento(Pagavel formaPagamento) {
11        this.formaPagamento = formaPagamento;
12    }
13
14    public void finalizar() {
15        formaPagamento.pagar(valorTotal);
16    }
17 }
```

Listing 10: Pedido guarda e delega para uma forma de pagamento

```
1 Pedido pedido = new Pedido(150.0, new Boleto("00190.00009"));
2
3 // O cliente muda de ideia antes de finalizar:
4 pedido.setFormaPagamento(new Pix("cliente@email.com"));
5
6 pedido.finalizar();
```



```
7 // "Transferindo R$ 150.0 via Pix cliente@email.com"
```

Listing 11: Trocando a estratégia em tempo de execução

O mesmo objeto `Pedido` mudou de comportamento em tempo de execução, apenas trocando **qual implementação** de `Pagavel` ele usa internamente – sem nenhum `if` para verificar o tipo.

2.3 O padrão Strategy

Definição

Strategy é um padrão de projeto em que um objeto (o **contexto**) delega uma tarefa a outro objeto, escolhido através de uma **interface comum** – e essa escolha pode ser trocada em tempo de execução.

No nosso exemplo: `Pedido` é o **contexto**; `Pagavel` é a **interface da estratégia**; `Boleto`, `CartaoCredito` e `Pix` são as **estratégias concretas**.

2.4 Strategy no dia a dia: Comparator

Você já usou Strategy sem perceber, lá na interface `List`:

```
1 List<String> nomes = new ArrayList<>();
2 nomes.add("Rex");
3 nomes.add("Bidu");
4 nomes.add("Amy");
5
6 Collections.sort(nomes); // ordem alfabetica padrao
7
8 Collections.sort(nomes, Comparator.comparing(String::length));
9 // agora ordena por tamanho do nome
```

Listing 12: Comparator como estratégia de ordenação

`Comparator` é uma interface: o contrato de "como comparar dois elementos". `Collections.sort` é o **contexto**. Cada `Comparator` diferente representa uma **estratégia de ordenação** diferente.

Parte 3: Por que Strategy é “Polimorfismo por Composição”

3.1 Dois jeitos de conseguir polimorfismo

Polimorfismo é quando a **mesma chamada de método** produz comportamentos diferentes, dependendo do objeto real por trás da referência. Até aqui, vimos duas técnicas completamente diferentes chegando exatamente nesse mesmo efeito.

3.2 Polimorfismo por herança

No exemplo clássico de `Jogador`, `JogadorNoob` e `JogadorVeterano`, uma variável do tipo `Jogador` pode apontar para qualquer subclasse; chamar `calcularXp()` executa um código

diferente dependendo de **qual classe** o objeto é.

A ideia central da herança

Para mudar o comportamento, você precisa mudar **o tipo do próprio objeto** – e, como vimos, o tipo de um objeto não pode mudar depois que ele foi criado.

3.3 Polimorfismo por composição

No exemplo de `Pedido` e `Pagavel`, `Pedido` é **sempre** a mesma classe. Ele guarda uma referência (um atributo) para um `Pagavel`. Chamar `formaPagamento.pagar()` executa um código diferente dependendo de **qual objeto** o atributo aponta – não do tipo de `Pedido`.

Polimorfismo por composição

É a técnica de obter comportamentos diferentes **sem** variar o tipo do objeto principal (isso seria herança), mas variando **qual objeto ele guarda internamente**, atrás de uma interface.

Strategy troca “que tipo de objeto isso é” por “o que esse objeto guarda dentro dele”.

Repare que isso é exatamente o mesmo princípio da Parte 1.10 (programar para a abstração): o `Pedido` depende apenas do contrato `Pagavel`, nunca de uma classe concreta. Strategy é, na prática, a aplicação sistemática desse princípio para resolver comportamento que muda em tempo de execução.

3.4 Um esclarecimento importante sobre a palavra “composição”

Cuidado com o sentido da palavra

Se vocês já estudaram associação, agregação e composição em UML, aqui a palavra “composição” está sendo usada em um sentido **mais amplo**, comum na literatura de padrões de projeto: “montar comportamento colocando um objeto *dentro* de outro através de um atributo” – em oposição a herança. Nesse sentido amplo, tanto associação quanto agregação quanto composição (no sentido estrito de UML) contam como “composição de objetos”.

No sentido **estrito** que vimos antes (o todo *cria* a parte dentro do próprio construtor, e a parte não sobrevive fora dele), a relação entre `Pedido` e `Pagavel` é, na verdade, mais próxima de uma **agregação**: o `Pagavel` é criado **fora** do `Pedido` e apenas entregue a ele pelo construtor ou pelo `setFormaPagamento`.

Como não se confundir

Ouviu “composição” falando de **padrões de projeto** (como em “favoreça composição em vez de herança”? Pense em “ter um objeto lá dentro, de qualquer jeito”. Ouviu “composição” falando de **UML/POO clássica**? Pense na relação forte e específica (o todo cria e é dono da parte).

Verifique seu entendimento

- No polimorfismo por herança, o que precisa mudar para o comportamento mudar? E no polimorfismo por composição?
- Por que a relação entre `Pedido` e `Pagavel` é uma agregação, e não uma composição, no sentido estrito de UML?

Parte 4: Passo a passo – desfazendo herança e inserindo Strategy

4.1 O roteiro geral

- Passo 1.** Identifique o método que **varia** entre as subclasses (o método abstrato que cada uma sobreescreve).
- Passo 2.** Crie uma **interface nova**, contendo apenas esse método.
- Passo 3.** Transforme cada subclasse em uma classe que **implementa** essa interface, mantendo apenas a fórmula/lógica que era exclusiva dela.
- Passo 4.** Elimine a hierarquia antiga: a classe original vira **uma única classe concreta**, sem subclasses de "categoria".
- Passo 5.** Adicione, na classe única, um **atributo do tipo da nova interface**, com um construtor que o recebe e um *setter* que permite trocá-lo depois.
- Passo 6.** Troque o corpo do método antigo por uma simples **delegação** para esse atributo.
- Passo 7.** Atualize todo lugar que criava as antigas subclasses, para em vez disso **passar a estratégia correta** no construtor da classe única.

4.2 Aplicando o roteiro: o problema do Jogador

Um jogo calcula a experiência (XP) ganha pelos jogadores ao final de cada partida. Jogadores **Noob** ganham XP direto; jogadores **Veterano** ganham XP multiplicado, mais um bônus. A modelagem inicial, com herança, é:

```
1 public abstract class Jogador {
2     private String nome;
3     private double pontosBase;
4
5     public Jogador(String nome, double pontosBase) {
6         this.nome = nome;
7         this.pontosBase = pontosBase;
8     }
9
10    public double getPontosBase() { return pontosBase; }
11
12    public abstract double calcularXp();
13 }
```

```

14
15 public class JogadorNoob extends Jogador {
16     public JogadorNoob(String nome, double pontosBase) {
17         super(nome, pontosBase);
18     }
19
20     @Override
21     public double calcularXp() {
22         return getPontosBase() * 1.0;
23     }
24 }
25
26 public class JogadorVeterano extends Jogador {
27     private double bonusVeterano;
28
29     public JogadorVeterano(String nome, double pontosBase, double
bonusVeterano) {
30         super(nome, pontosBase);
31         this.bonusVeterano = bonusVeterano;
32     }
33
34     @Override
35     public double calcularXp() {
36         return (getPontosBase() * 2.5) + bonusVeterano;
37     }
38 }

```

Listing 13: Antes: hierarquia com JogadorNoob e JogadorVeterano

O problema aparece quando um jogador **evolui**: a variável **ana** foi declarada como **JogadorNoob**, e o tipo de um objeto em Java não pode ser alterado depois que ele foi criado. A única saída, com herança, é criar um objeto novo e copiar os dados manualmente – um processo trabalhoso, propenso a bugs silenciosos, e que deixa **duas versões** do jogador circulando pelo sistema.

4.3 Passo 1 e 2 – o que varia, e a nova interface

O que varia entre **JogadorNoob** e **JogadorVeterano** é **apenas** a fórmula dentro de **calcularXp()**. Isso vira uma interface nova:

```

1 public interface EstrategiaXp {
2     double calcular(double pontosBase);
3 }

```

Listing 14: A interface EstrategiaXp

4.4 Passo 3 – cada subclasse vira uma estratégia

```

1 public class EstrategiaXpNoob implements EstrategiaXp {
2     @Override
3     public double calcular(double pontosBase) {
4         return pontosBase * 1.0;

```

```

5     }
6 }
7
8 public class EstrategiaXpVeterano implements EstrategiaXp {
9     private double bonus;
10
11     public EstrategiaXpVeterano(double bonus) {
12         this.bonus = bonus;
13     }
14
15     @Override
16     public double calcular(double pontosBase) {
17         return (pontosBase * 2.5) + bonus;
18     }
19 }

```

Listing 15: As fórmulas migram para classes que implementam EstrategiaXp

A fórmula de cada categoria é exatamente a mesma de antes – só mudou **onde ela mora**.

4.5 E os métodos que eram herdados de graça?

Na hierarquia antiga, JogadorNoob e JogadorVeterano tinham acesso a `getPontosBase()` (e a qualquer outro método público de Jogador) **de graça**, simplesmente por causa do **extends**. Herança dá acesso automático a tudo que a superclasse oferece.

Agora preste atenção: `EstrategiaXpNoob` e `EstrategiaXpVeterano` não têm mais nenhuma relação de herança com `Jogador` – elas apenas implementam `EstrategiaXp`. Isso significa que elas **não herdam absolutamente nada** de `Jogador`: nem atributos, nem métodos.

Composição não herda nada de graça

Se o método da estratégia precisar de algum dado que antes vinha "de graça" pela herança (como `pontosBase`), esse dado precisa ser **passado explicitamente** como parâmetro do método da interface. É exatamente por isso que a assinatura é `calcular(double pontosBase)`, e não apenas `calcular()`.

```

1 // ANTES (heranca): JogadorVeterano ja tinha getPontosBase() de
  // graca
2 @Override
3 public double calcularXp() {
4     return (getPontosBase() * 2.5) + bonusVeterano;
5 }
6
7 // DEPOIS (composicao): EstrategiaXpVeterano nao tem getPontosBase
  // ();
8 // pontosBase precisa chegar como parametro do metodo da interface
9 @Override
10 public double calcular(double pontosBase) {
11     return (pontosBase * 2.5) + bonus;

```

12 }

Listing 16: O dado que antes era herdado agora é passado por parâmetro

Como decidir o que colocar na assinatura da interface

Ao desenhar o método da interface, pergunte: **de quais dados do contexto original essa lógica realmente precisa?** Coloque exatamente esses dados como parâmetros – nem mais, nem menos. Se a estratégia precisasse também do nome do jogador, a assinatura teria que crescer para `calcular(String nome, double pontosBase)`.

Uma alternativa: passar o objeto inteiro

Em vez de passar cada dado separadamente, também é possível passar o próprio `Jogador` como parâmetro (`calcular(Jogador jogador)`), e usar `jogador.getPontosBase()` dentro da estratégia. Isso evita ter que alterar a assinatura toda vez que surge uma nova necessidade, mas em troca acopla a estratégia à classe `Jogador`. A escolha entre "vários parâmetros pequenos" e "um parâmetro com o objeto inteiro" é uma decisão de projeto, não uma regra fixa.

4.6, 4.7 e 4.8 – unindo as classes, adicionando o atributo trocável e delegando

```
1 public class Jogador {
2     private String nome;
3     private double pontosBase;
4     private EstrategiaXp estrategiaXp;
5
6     public Jogador(String nome, double pontosBase, EstrategiaXp
7     estrategiaXp) {
8         this.nome = nome;
9         this.pontosBase = pontosBase;
10        this.estrategiaXp = estrategiaXp;
11    }
12
13    public void setEstrategiaXp(EstrategiaXp estrategiaXp) {
14        this.estrategiaXp = estrategiaXp;
15    }
16
17    public double calcularXp() {
18        return estrategiaXp.calcular(pontosBase);
19    }
20 }
```

Listing 17: Depois: uma única classe `Jogador`, com estratégia trocável

Não existe mais `JogadorNoob` nem `JogadorVeterano` – existe apenas `Jogador`, com uma estratégia trocável.

4.9 Passo 7 – atualizando os pontos de criação

```

1 Jogador ana = new Jogador("Ana", 100, new EstrategiaXpNoob());
2 System.out.println(ana.calcularXp()); // 100.0
3
4 // Ana evoluiu e virou veterana:
5 ana.setEstrategiaXp(new EstrategiaXpVeterano(50.0));
6 System.out.println(ana.calcularXp()); // 300.0

```

Listing 18: Ana evolui: apenas trocando a estratégia

`ana` é o **mesmo objeto** do início ao fim. Não criamos um objeto novo, não copiamos nenhum atributo, e qualquer outra parte do sistema que já tinha uma referência para `ana` vê a **mudança automaticamente**.

4.10 Resultado final

Papel	Antes (herança)	Depois (Strategy)
O que varia	O tipo do objeto	O atributo <code>estrategiaXp</code>
Trocar de categoria	Criar objeto novo, copiar dados	<code>setEstrategiaXp(...)</code>
Identidade do objeto	Perdida na troca	Preservada

Verifique seu entendimento

- No roteiro do Passo a Passo, o que exatamente vira a nova interface?
- Depois da refatoração, o que aconteceu com a classe abstrata `Jogador` original?
- Se o jogo ganhasse uma categoria `Lenda`, o que precisaria ser criado – e o que **não** precisaria mudar?

Recapitulando

- uma **interface** é um contrato: define quais métodos uma classe promete ter, sem dizer como implementá-los;
- herança representa "**é um**"; interface representa "**sabe fazer**";
- **programar para a abstração** significa depender da interface, não da classe concreta;
- **Strategy** é um contexto que delega uma tarefa a uma interface comum, permitindo trocar a estratégia em tempo de execução;
- composição não herda nada de graça: qualquer dado que a estratégia precisar do contexto original deve ser passado explicitamente como parâmetro do método da interface;
- Strategy é chamado de **polimorfismo por composição** porque troca o comportamento variando **o que o objeto guarda dentro dele**, e não o tipo do próprio objeto;

- o roteiro para substituir herança por Strategy sempre segue os mesmos passos: extrair o método que varia para uma interface, transformar cada subclasse em uma estratégia concreta, unir as classes originais em uma só, e adicionar a estratégia como um atributo trocável.